

PROJECT NAME HERE

Author	Name of author(s)
Reviewer	Name of reviewer(s)
Circulation	Confidential, for internal use only.
Project Name	Short name for project.
Description	One line description of project.

Detailed History of Changes:

Version	Date	Author	Description of Changes

Contents

- 1 Scope
 - 1.1 Identification
 - 1.2 System overview
 - 1.3 Document overview
- 2 Referenced documents
- 3 System-wide design decisions
- 4 Architectural design
 - 4.1 Components
 - 4.2 Concept of execution
 - 4.3 Interface design
 - 4.3.1 Interface identification
 - 4.3.2 (Identifier of interface)
- 5 Requirements traceability
- 6 Checklist
- 7 Notes
- A Appendixes

1 Scope

This section shall be divided into the following paragraphs.

1.1 Identification

This paragraph shall contain a full identification of the system and the software to which this document applies, including, as applicable, identification number(s) and name(s), title(s), abbreviation(s), version number(s), and release number(s).

1.2 System overview

This paragraph shall briefly state the purpose of the system and the software to which this document applies. It shall describe the general nature of the system and software; summarize the history of system development, operation, and maintenance; identify the project sponsor, and developer; and list other relevant documents.

1.3 Document overview

This paragraph shall summarize the purpose and contents of this document and shall describe any security or privacy considerations associated with its use.

2 Referenced documents

This section shall list the number, title, revision, and date of all documents referenced in this document (for example, internal documents such as functional specifications, marketing requirements documents and investigation reports, and external documents such as reference guides, user manuals, etc.).

3 System-wide design decisions

This section shall be divided into paragraphs as needed to present system-wide design decisions, that is, decisions about the system's behavioral design (how it will behave, from a user's point of view, in meeting its requirements, ignoring internal implementation) and other decisions affecting the selection and design of the components that make up the system. If all such decisions are explicit in the requirements or are deferred to the design of the software units, this section shall so state. Design decisions that respond to requirements deemed critical, such as those for safety, security, or privacy, shall be placed in separate subparagraphs. Examples of system-wide design decisions are the following:

1. Design decisions regarding inputs the system will accept and outputs it will produce, including interfaces with other systems, hardware, software, and users (4.3.x of this document identifies topics to be considered in this description).

2. Design decisions on system behavior in response to each input or condition, including actions to perform, response times and other performance characteristics, description of physical systems modelled, selected equations/algorithms/rules, and handling of disallowed inputs or conditions.
3. Design decisions on how databases/data files will appear to the user (4.3.x of this document identifies topics to be considered in this description).
4. Selected approach to meeting safety, security, and privacy requirements.
5. Other system-wide design decisions made in response to requirements, such as selected approach to providing required flexibility, availability, and maintainability.

4 Architectural design

This section shall be divided into the following paragraphs to describe the architectural design.

4.1 Components

This paragraph shall:

1. Identify the software units that make up the system.
Note: A software unit is an element in the design of a system; for example, a major subdivision of a system, a component of that subdivision, a class, object, module, function, routine or database. Software units may occur at different levels of a hierarchy and may consist of other software units. Software units in the design may or may not have a one-to-one relationship with the code and data entities (routines, procedures, databases, data files, etc.) that implement them or with the computer files containing those entities.
2. Show the static (“consists of”) relationship(s) of the software units. Multiple relationships may be presented, depending on the selected software design methodology (for example, in an object-oriented design, this paragraph may present the class and object structures as well as the modules and process architectures).
3. State the purpose of each software unit and identify the requirements and system design decisions allocated to it. (Alternatively, the allocation of requirements may be provided in Section 5).
4. Identify each software unit’s development status/type (such as new development, existing design or software to be reused as is, existing design or software to be reengineered, software to be developed for reuse, etc.) For existing design or software, the description shall provide identifying information, such as name, version, documentation references, library, etc.
5. Describe planned utilization of computer hardware resources (such as processor capacity, auxiliary storage capacity, and communications/network equipment capacity), if appropriate.
6. Identify the program library in which the software that implements each software unit is to be placed.

4.2 Concept of execution

This paragraph shall describe the concept of execution among the software units. It shall include diagrams and descriptions showing the dynamic relationship of the software units, that is, how they will interact during operation, including, as applicable:

- Flow of execution control,
- Data flow,
- Dynamically controlled sequencing,
- State transition diagrams,
- Timing diagrams,
- Priorities among units,
- Handling of interrupts and signals,
- Timing/sequencing relationships,
- Exception and error handling,
- Concurrent execution,
- Dynamic allocation/deallocation of memory or other resources,
- Dynamic creation/deletion of objects,
- Processes,
- Tasks, and
- Other aspects of dynamic behavior.

4.3 Interface design

This paragraph shall be divided into the following subparagraphs to describe the interface characteristics of the software units. It shall include both interfaces among the software units and their interfaces with external entities such as systems and users. If part or all of this information is described elsewhere, these sources may be referenced.

4.3.1 Interface identification

This paragraph shall identify each interface to the software. One or more interface diagrams shall be provided, as appropriate, to depict the interfaces.

4.3.2 (Identifier of interface)

This paragraph (beginning with 4.3.2) shall identify an interface, briefly identify the interfacing entities, and shall be divided into subparagraphs as needed to describe the interface characteristics of one or both the interfacing entities. If a given interfacing entity is not covered by this document (for example, an external system or user) but its interface characteristics need to be mentioned to describe interfacing entities that are, these characteristics shall be stated as assumptions or as ``When [the entity not covered] does this, [the entity that is covered] will ..." This paragraph may reference other documents (such as data dictionaries, standards for protocols, and standards for user interfaces) in place of stating the information here. The design description shall include the following, as applicable, presented in any order suited to the information to be provided, and shall note any differences in these characteristics from the point of view of the interfacing entities (such as

different expectations about the size, frequency, or other characteristics of data elements):

1. Priority assigned to the interface by interfacing entities
2. Type of interface (such as real-time data transfer, storage of data, command-line user interface, etc.) to be implemented
3. Characteristics of individual data elements that interfacing entity(ies) will provide, store, send, access, receive, etc., such as:
 - Technical name (e.g., variable or field name in code or database), non-technical name, or other identifier
 - Data type (alphanumeric, integer, etc.)
 - Size and format
 - Units of measurement
 - Range or enumeration of possible values
 - Accuracy and precision
 - Priority, timing, frequency, volume, sequencing, and other constraints
 - Security and privacy constraints
 - Sources (setting/sending entities) and recipients (using/receiving entities)
4. Characteristics of data element assemblies (records, messages, files, arrays, displays, reports, etc.) that the interfacing entity(ies) will provide, store, send, access, receive, etc., such as:
 - Technical name (e.g., record or data structure name in code or database), non-technical name, or other identifier
 - Data elements in the assembly and their structure (number, order, grouping)
 - Medium (such as disk) and structure of data elements/assemblies on the medium
 - Visual and auditory characteristics of displays and other outputs (such as colours, layouts, fonts, icons and other display elements, beeps, lights)
 - Relationships among assemblies, such as sorting/access characteristics
 - Priority, timing, frequency, volume, sequencing, and other constraints
 - Security and privacy constraints
 - Sources (setting/sending entities) and recipients (using/receiving entities)
5. Characteristics of communication methods that the interfacing entity(ies) will use for interface, such as:
 - Identifier
 - Communication links/bands/frequencies/media and their characteristics
 - Message formatting
 - Flow control (such as sequence numbering and buffer allocation)
 - Data transfer rate, whether periodic/asynchronous, and interval between transfers
 - Routing, addressing, and naming conventions
 - Transmission services, including priority and grade
 - Safety/security/privacy considerations, such as encryption, user authentication, etc.
6. Characteristics of protocols the interfacing entity(ies) will use for the interface, such as:
 - Identifier
 - Priority/layer of the protocol
 - Packeting, including fragmentation and reassembly, routing, and addressing
 - Legality checks, error control, and recovery procedures
 - Synchronization, including connection establishment, maintenance, termination
7. Other characteristics

5 Requirements traceability

This section shall contain:

1. Traceability from each software unit in this document to the requirements allocated to it. (Alternatively, this traceability may be provided in [4.1.](#))
2. Traceability from each requirement to the software units to which it is allocated.

6 Checklist

To avoid loose ends and ensure project quality, check the following questions while you start low-level design and implementation.

- Are both Unix and NT installer updated for this project?
- Does the implementation consider NT?
- Have you arranged code review for this project?

7 Notes

This section shall contain any general information that aids in understanding this document (e.g., background information, glossary, rationale). This section shall include an alphabetical listing of all acronyms, abbreviations, and their meanings as used in this document and a list of any terms and definitions needed to understand this document.

A Appendixes

Appendixes may be used to provide information published separately for convenience in document maintenance (e.g., charts). As applicable, each appendix shall be referenced in the main body of the document where the data would normally have been provided. Appendixes shall be lettered alphabetically (A, B, etc.).